アカウント技術入門

これで君もSaaSマスターだ！



https://sql-playground-beta.vercel.app/events/kc3-2026

しおまち / Shiomachi

本名：丸山拓真

・未踏ジュニア’22 修了

・SecHack365’23 修了



勉強会

2025.08.21

Webアプリ開発のためのプログラミング言語処理系入門！



ポスト
シェア
B! はてブ
送る
Pocket

勉強会概要



今日話すこと

そもそも
「アカウント」ってなんだ



いろんなサービスに
アカウントという概念はある

そもそも「アカウントってなんだ？」

「あなたはだれ」を識別して
データを紐付けるためのもの

そもそも「アカウントってなんだ？」

ユーザの履歴
視聴履歴 / 購入履歴

「誰がみた」？
ユーザの行動に基づいて推薦

ユーザが送信したデータ
送信したメッセージ / 画像

「誰が送ったメッセージ」？
「誰の写真」？

ユーザが受信したデータ
メールボックス

「誰に見せていいデータ」？

他人に勝手に見せちゃいけないデータを
記録 / 表示する時に必要になる

そもそも「アカウントってなんだ？」

今回はアカウント機能を持つ
SaaSを開発することを想定

今回のテーマ

藩士向けSaaSを作ってみよう！

そのデータベース設計をしてみよう



今回のテーマ



藩士が利用できる掲示板サービス

・所属する藩の藩士が書き込んだものは見える
・別の藩のメッセージは見えない
・匿名ではない（誰が書いたかはわかる）

今回はこれのDB設計をしていく

今回のテーマ

藩士が利用できる掲示板サービス

最低限必要そうなもの

・メッセージを見る
・メッセージを書き込む
・誰が書いたメッセージなのかを紐づけ

次の仕組みを考えよう！

・「アカウント」をDBに作る
・所属先のデータと結びつける
・ログイン時に本人を確認する

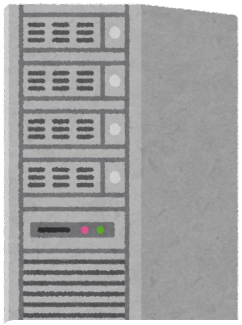


今回のテーマ

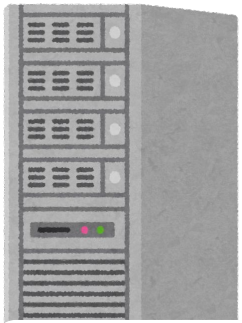
12	#59	[応用] 複合インデックスでメッセージ検索を高速化する
13	#60	[応用] インデックス列順序を検索条件に合わせる
14	#61	[応用] Covering Indexで一覧取得を完結させる
15	#62	[応用] 複合主キーで藩の所属を一意にする
16	#63	[応用] JOINでもテナント境界を守る
17	#64	[応用] 複合外部キーでテナント境界を守る
18	#65	[応用] UNIQUE制約をテナント単位に設計する
19	#66	[応用] JOIN集計とHAVINGで活発な藩を探す
20	#67	[応用] 複合インデックスの基本を身につける

応用問題もあるよ！

どこまで共通化するか？



DBサーバー



アプリケーション
サーバー



藩 = テナント





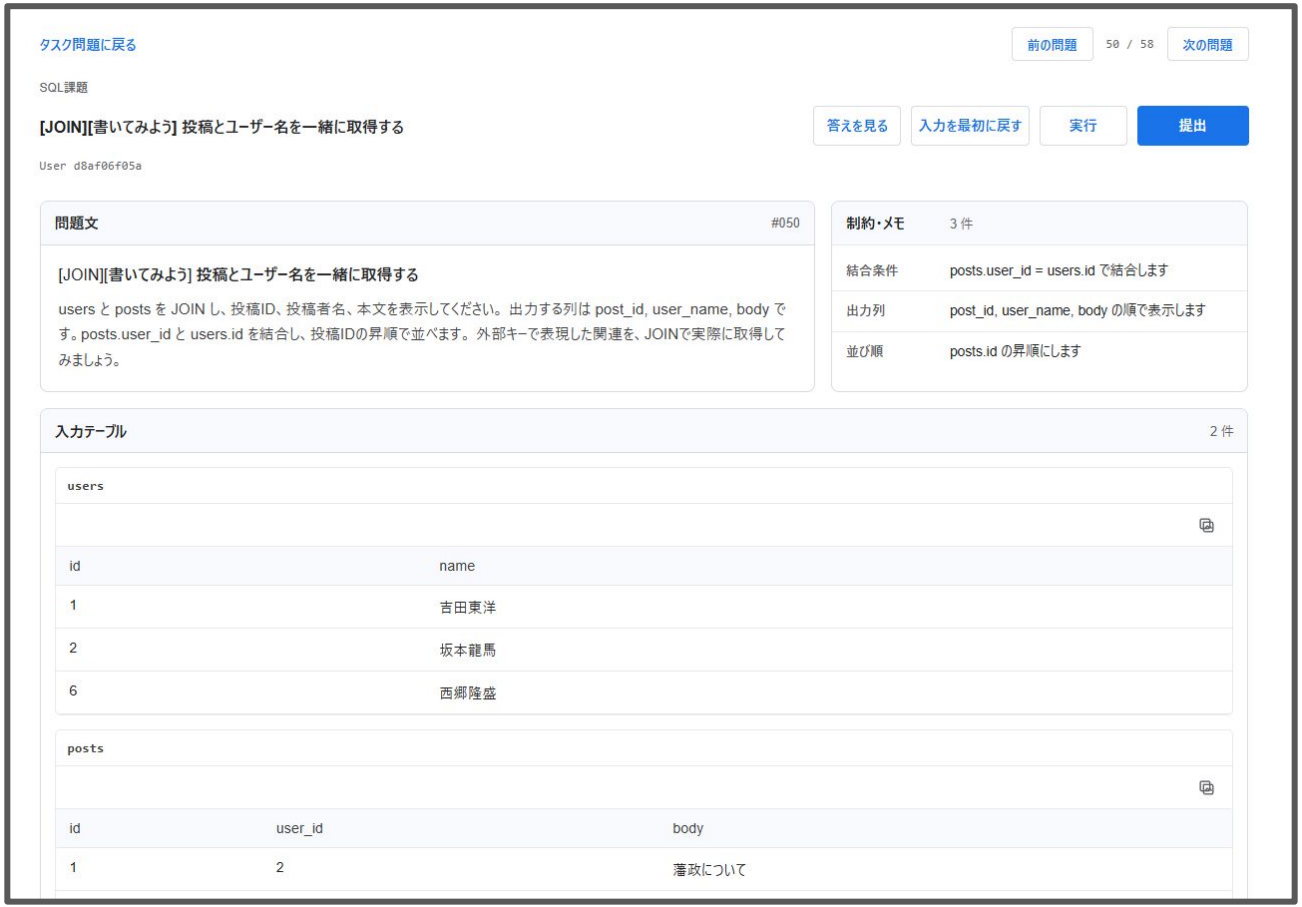
メッセージ

メッセージ





今回使うツール



タスク問題に戻る

前の問題 50 / 58 次の問題

SQL課題

[JOIN][書いてみよう] 投稿とユーザー名を一緒に取得する

答えを見る 入力を最初に戻す 実行 提出

User d8af06f05a

問題文 #050

[JOIN][書いてみよう] 投稿とユーザー名を一緒に取得する

users と posts を JOIN し、投稿ID、投稿者名、本文を表示してください。出力する列は post_id, user_name, body です。posts.user_id と users.id を結合し、投稿IDの昇順で並べます。外部キーで表現した関連を、JOINで実際に取得してみましょう。

制約・メモ 3 件

結合条件	posts.user_id = users.id で結合します
出力列	post_id, user_name, body の順で表示します
並び順	posts.id の昇順にします

入力テーブル 2 件

users

id	name
1	吉田東洋
2	坂本龍馬
6	西郷隆盛

posts

id	user_id	body
1	2	藩政について

進めかた

登壇者はスライドをもとに話します

演習問題は話している内容とリンクしているので、
随時聞きながら、解いていただければ！

眠たくなったらお昼寝OKです（私のLT・勉強会では慣　例！）
・あとから資料配布します。演習問題も後から解けます

離席自由



データベースで「ユーザー」を表現しよう（一般的な形）

#48[テーブル設計][書いてみよう]
users テーブルを作成する

データベースでユーザを表現する

実はこれは簡単

データベースにユーザのIDを持っておけばいい



その操作に必要な

SQL DDL/DML入門

SQLテーブルを作る時には
Data Definition Language（DDL）を用いる

DDL入門

テーブル名

```
CREATE TABLE users (
  id BIGINT PRIMARY KEY
);
```

```
CREATE TABLE users (
  id BIGINT PRIMARY KEY
)
```

カラム名

DDL入門

```
CREATE TABLE users (
  id BIGINT PRIMARY KEY
);
```

型

DDL入門

```
CREATE TABLE users (
  id BIGINT PRIMARY KEY
);
```

主キー

DDL入門

```
CREATE TABLE users (
  id BIGINT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(255),
  password_digest VARCHAR(255)
);
```

ユーザーID / 名前 / メール / パスワード

DDL入門

id	name	email	password_digest

ユーザーID / 名前 / メール / パスワードを作成できた！

SQLテーブルを操作する時には Data Manipulation Language（DML）を用いる

DML入門

```
INSERT INTO users (id, name, email, password_digest)
VALUES (1, '坂本龍馬', 'sakamoto@tosahan.example.com',
'$2b$12$k9L0G8YQz4jz4mYQJ6qY7uJ8cNq6x5E9mYwQWqY8nC2
N3pLxY6QxK');
```

よくある
SELECT / INSERT / UPDATE / DELETE

DDL入門

実行結果 3 行

id	name	email	password_digest
1	坂本龍馬	sakamoto@tosahan.exsample.com	$2b$12$k9L0G8YQz4jz4mYQJ6qY7uJ8cNq6x5E9mYwQWqY8nC2N3pLxY6QxK
2	中岡慎太郎	nakaoka@tosahan.exsample.com	$2b$12$uJ3V8WmN0aP7K4sD2hQ9EeGq6kH8vT5yM1xC7rN4bF9zL2wQ8pS6K
3	武市半平太	takechi@tosahan.exsample.com	$2b$12$A7mQ2kL9vX4cN6pT1sE8OuY3rJ5hG0wF9bD2xZ6nK4qM8cP1vR7eS

ユーザーID / 名前 / メール / パスワードを格納できた！

DML入門

```
CREATE TABLE posts (
  id BIGINT PRIMARY KEY,
  user_id BIGINT,
  body VARCHAR(255),

  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

ついでにメッセージテーブルも作ってみる

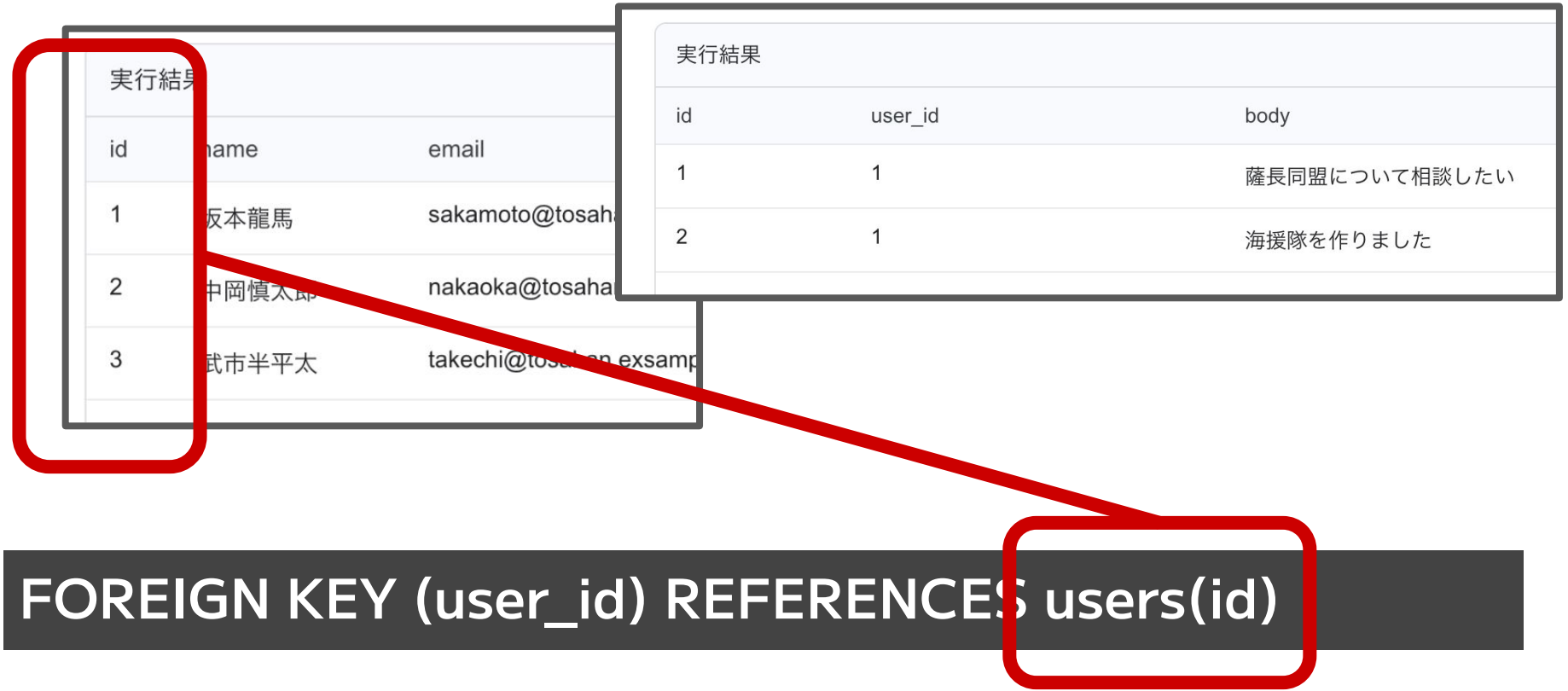
#49 [外部キー][書いてみよう] 投稿をユーザーに紐付ける

外部キー

```
FOREIGN KEY (user_id) REFERENCES users(id)
```

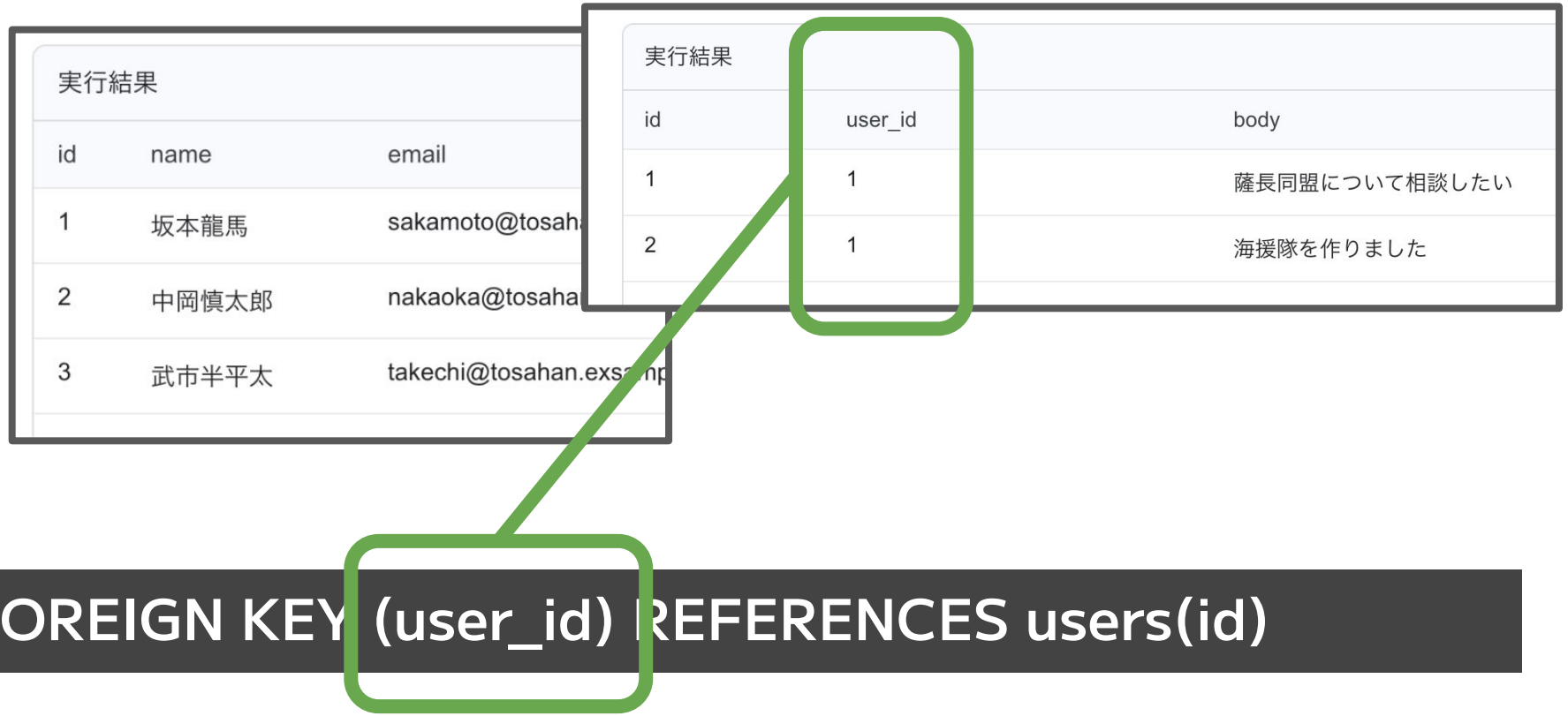
データベースのテーブルとテーブルを紐づける

データを紐付けていこう（外部キー）

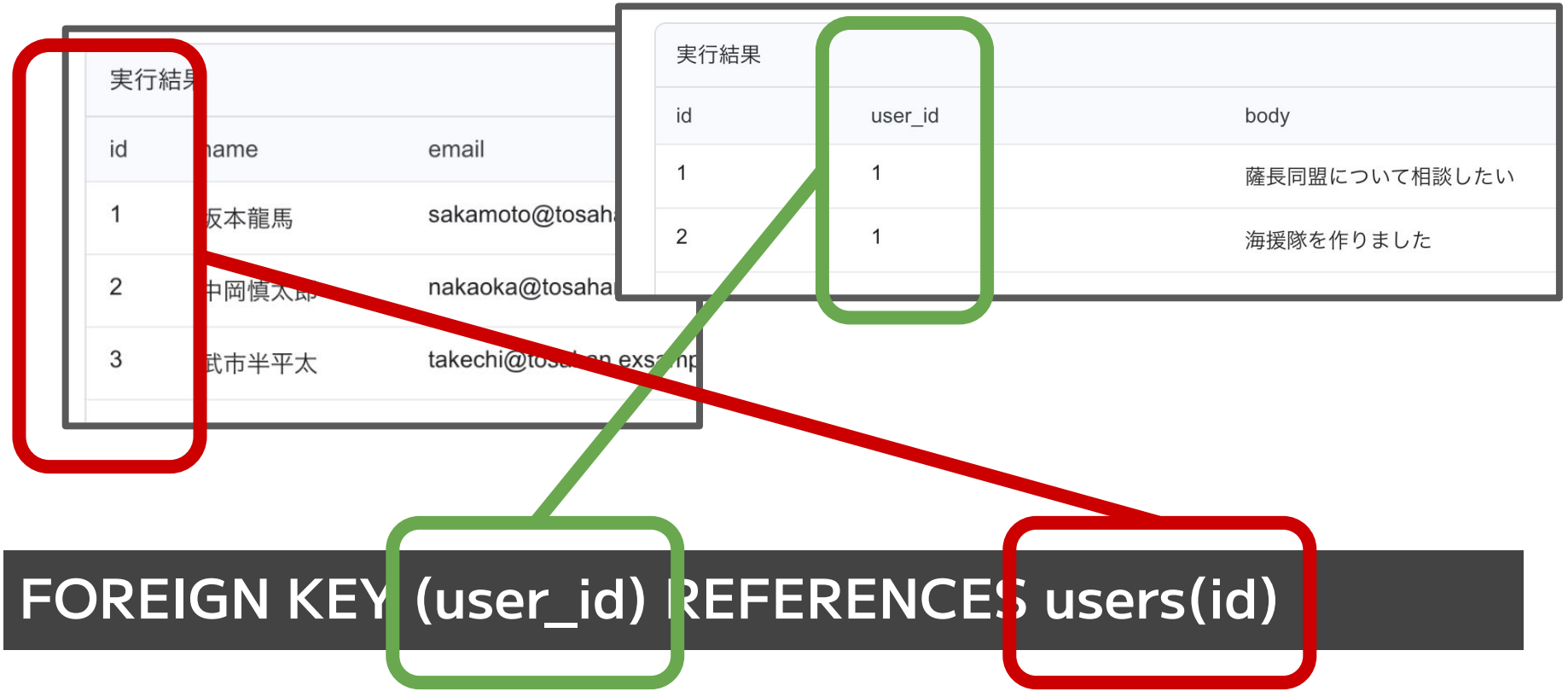



```
FOREIGN KEY (user_id) REFERENCES users(id)
```

データを紐付けていこう（外部キー）



データを紐付けていこう（外部キー）



データを紐付けていこう（外部キー）

usersテーブルとpostsテーブルを紐付けることができた！

実行結果

id	name	body
1	坂本龍馬	海援隊を作りました
1	坂本龍馬	薩長同盟について相談したい
2	中岡慎太郎	陸援隊について話したい
3	武市半平太	土佐勤王党について相談したい

JOINすると連結できる

#50 [JOIN][書いてみよう] 投稿とユーザー名を一緒に取得する

データを紐付けていこう（外部キー）

= 書き込みの時、users.idに存在しないidは posts.user_idに使用できない！

usersテーブル

実行結果

id	name	email
1	坂本龍馬	sakamoto@tosahan.e
2	中岡慎太郎	nakaoka@tosahan.ex
3	武市半平太	takechi@tosahan.exs

postsテーブル

実行結果

id	user_id	body
1	1	薩長同盟について相談したい
2	1	海援隊を作りました

アカウント技術入門

めでたし　めでたし



アカウント技術入門

めでたし　めでた

そうはいかない



そうはいかない・・・

現代のシステムは複雑多岐
さまざまなニーズが出てきた

会社・団体単位で情報を管理した
膨大なデータを捌く必要がでてきた
特定の用途への認証・認可方式がでてきた
パスワードレス認証 / OAuth 2.0 / OpenID Connect

どこまでいっても それらを 支えるのは
データベース設計

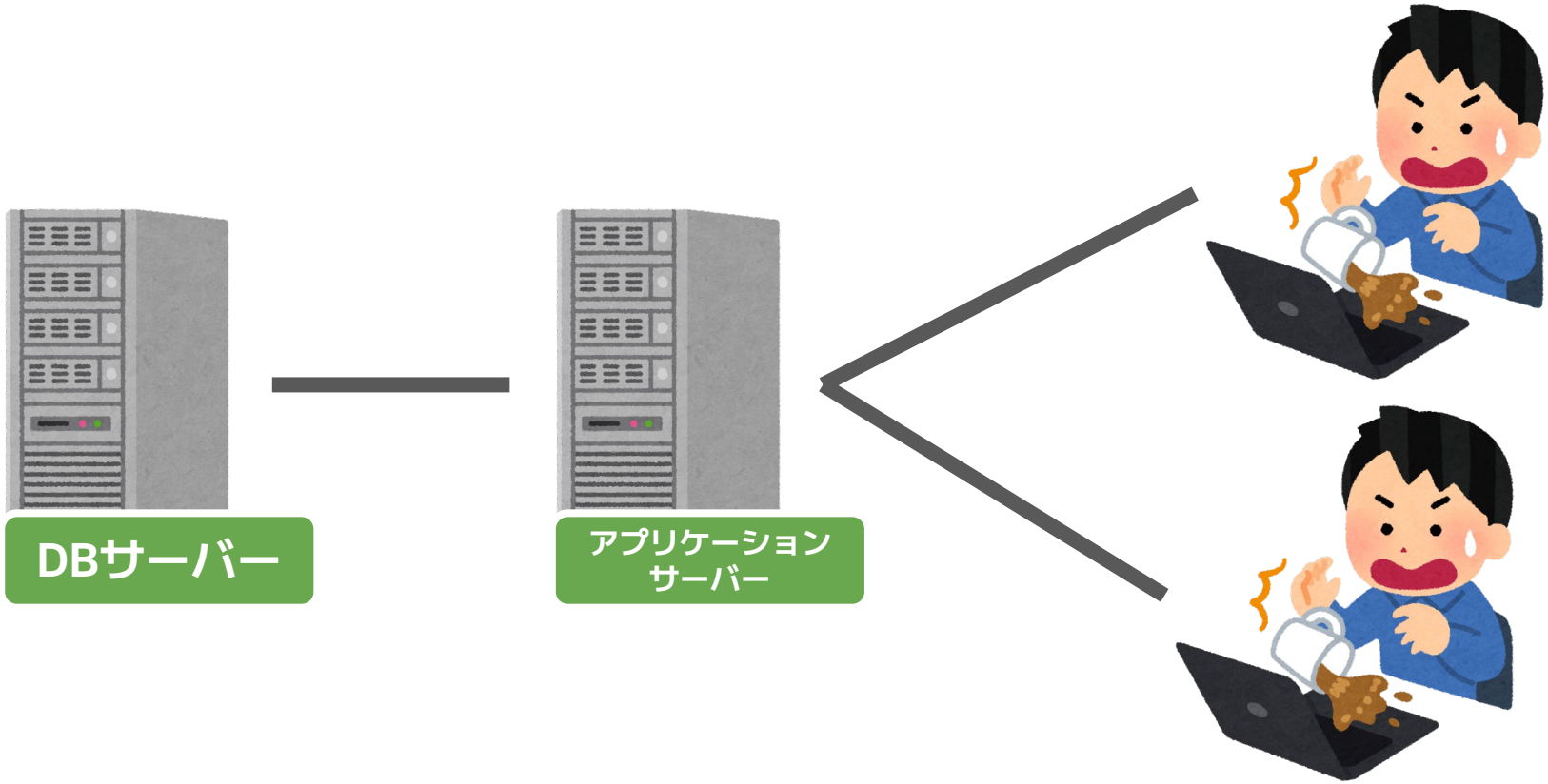


マルチテナント

マルチテナントとは

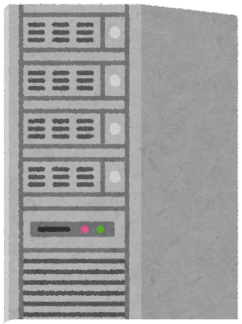
1つのアプリケーションを
複数の企業顧客が使える仕組み

一般的なWebサービス

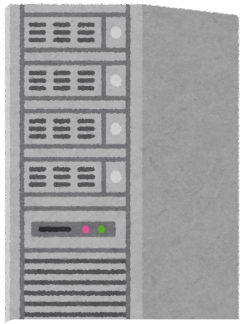


顧客ごとの専用環境（シングルテナント）





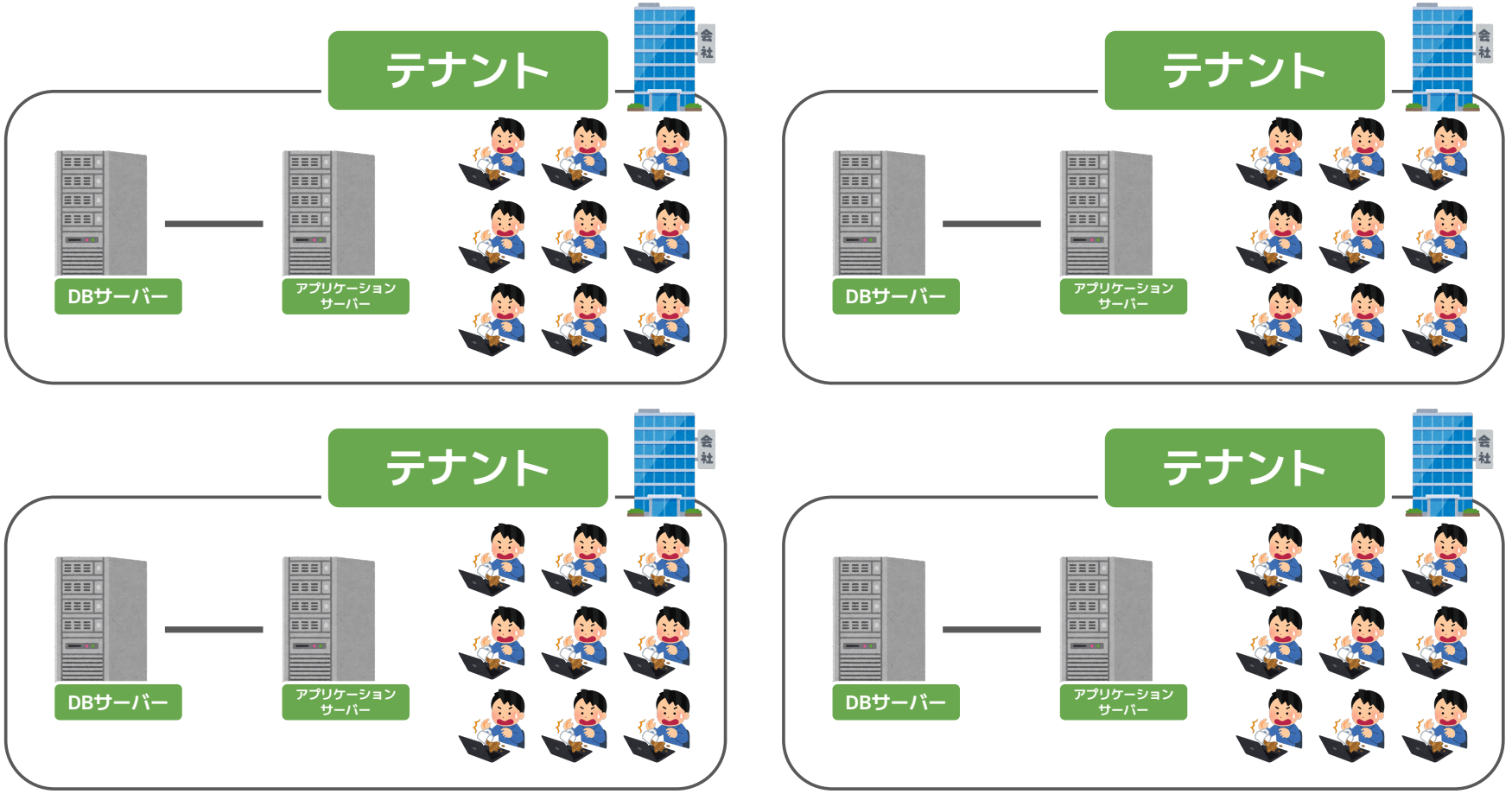
DBサーバー



アプリケーション
サーバー



顧客ごとの専用環境（シングルテナント）



マルチテナントとは

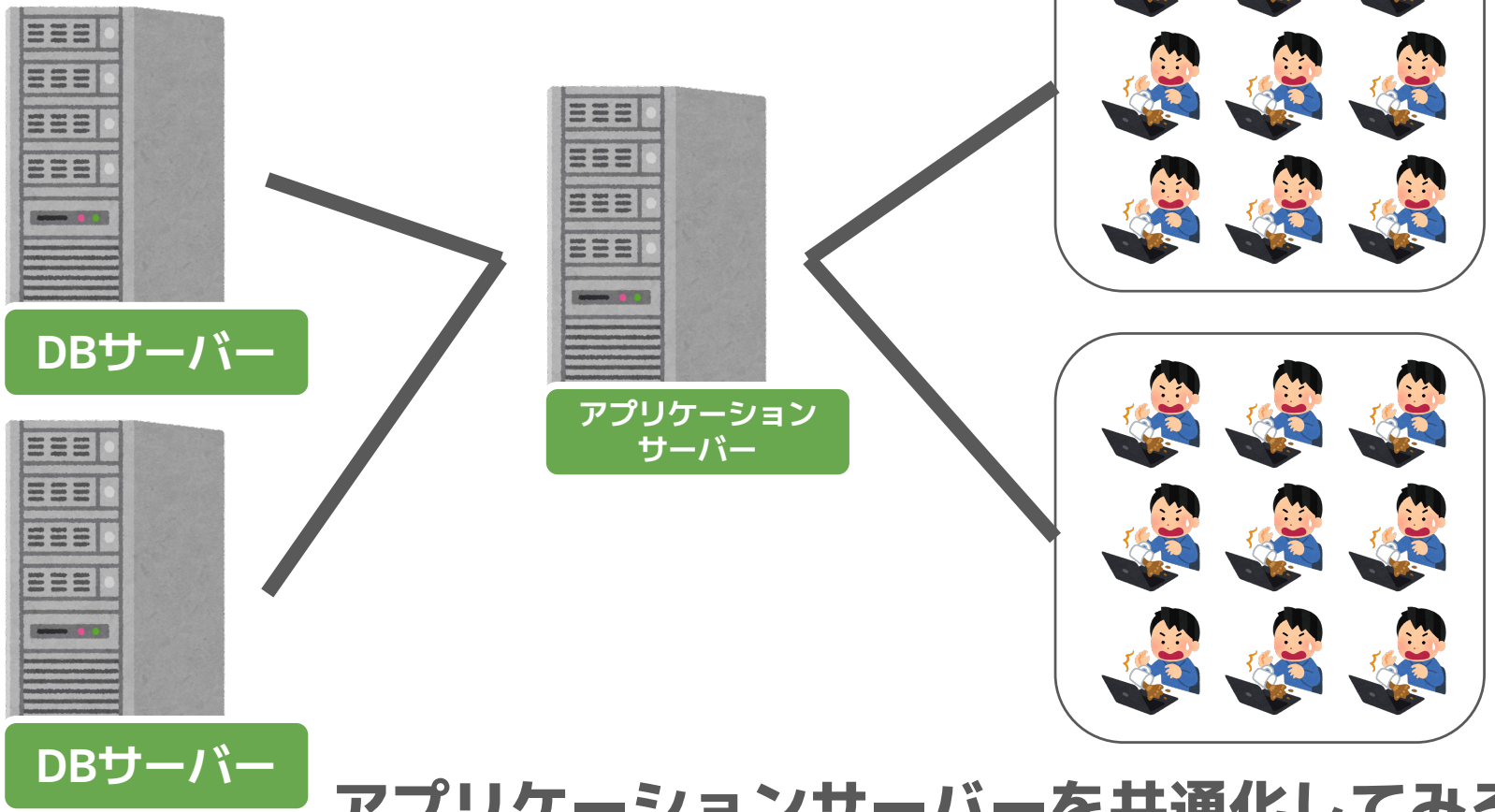
シングルテナント型の提供は
立ち上げにコストが・・・

マルチテナントとは

どこかを共通化できない？

管理コスト / 立ち上げコストを抑えたい・・

どこまで共通化するか？

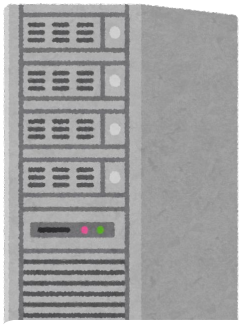


アプリケーションサーバーを共通化してみる

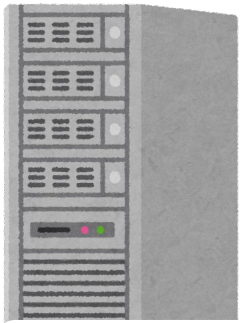
マルチテナントとは

いっそのことDBまで一緒に・・・

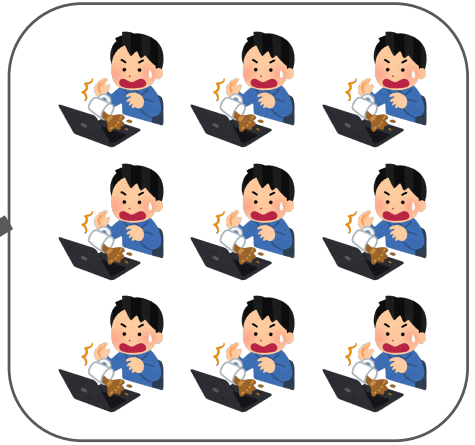
どこまで共通化するか？

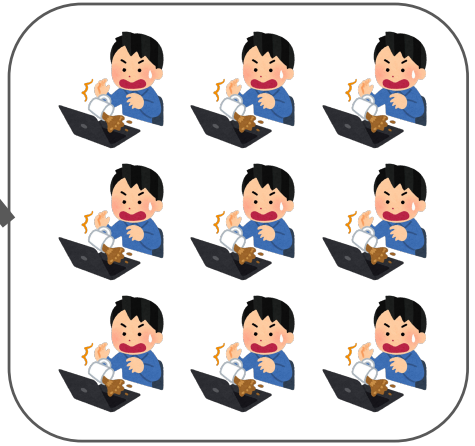


DBサーバー



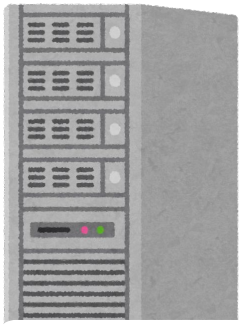
アプリケーション
サーバー



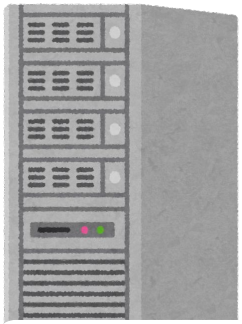


よさそう

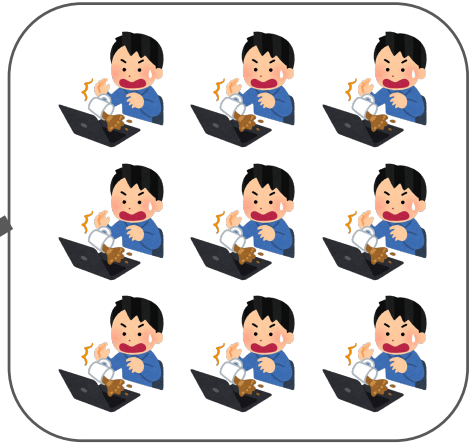
どこまで共通化するか？

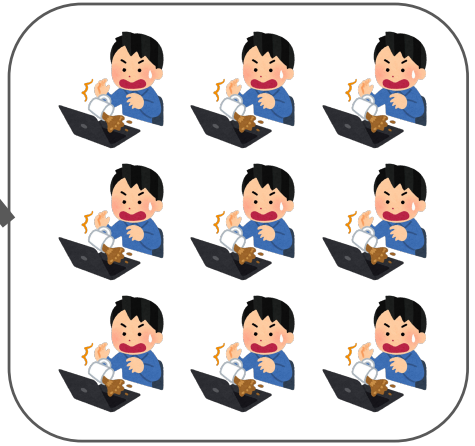


DBサーバー



アプリケーション
サーバー





企業どうしのデータを
どう分離するか？

マルチテナントとは

そもそもデータベースまで共通化とは・・・？

マルチテナントとは

同じテーブルに
複数の企業の情報をのっけてしまう

マルチテナントとは

そんなことが！？

マルチテナントとは

理屈上はできますよね・・・

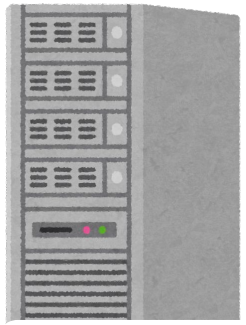
マルチテナントとは

id	tenant_id	role	name
1	1	家老	吉田東洋
2	1	郷士	坂本龍馬
3	1	藩士	後藤象二郎
4	1	藩士	武市半平太
5	2	家老	小松帯刀
6	2	藩士	西郷隆盛
7	2	藩士	大久保利通
8	3	家老	木戸孝允
9	3	藩士	高杉晋作
10	3	藩士	伊藤博文

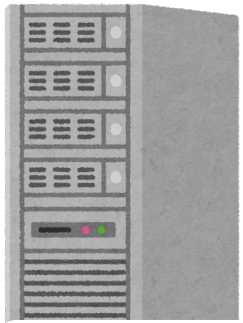
id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります

わかりやすいよう正規化してないです

どこまで共通化するか？

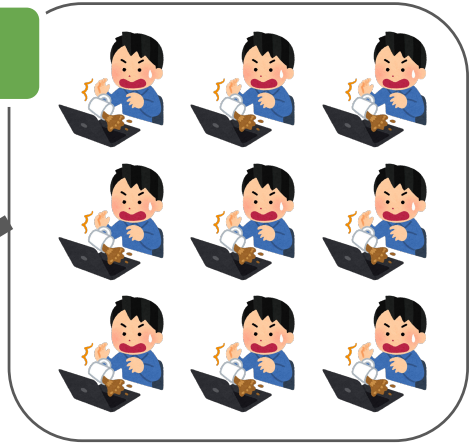


DBサーバー



アプリケーション
サーバー

テナント





テナント

マルチテナントシステム

テナント単位で情報をとってくる

テナントに属するテーブルには tenant_id を持たせる

usersテーブル

id	tenant_id	role	name
1	1	家老	吉田東洋
2	1	郷士	坂本龍馬
3	1	藩士	後藤象二郎
4	1	藩士	武市半平太
5	2	家老	小松帯刀
6	2	藩士	西郷隆盛
7	2	藩士	大久保利通
8	3	家老	木戸孝允
9	3	藩士	高杉晋作
10	3	藩士	伊藤博文

postsテーブル

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります

#51 [マルチテナント][書いてみよう] データに tenant_id を持たせる

マルチテナントとは

id	name	tenant	role
1	坂本龍馬	土佐藩	郷士
2	後藤象二郎	土佐藩	藩士
3	武市半平太	土佐藩	藩士
4	西郷隆盛	薩摩藩	藩士
5	大久保利通	薩摩藩	藩士
6	小松帯刀	薩摩藩	家老
7	木戸孝允	長州藩	藩士
8	高杉晋作	長州藩	藩士
9	吉田松陰	長州藩	藩士



テナントをまたいで情報が漏れるとかなりまずい
（特に薩長同盟以前は）

テナント単位で情報をとってくる

id	tenant_id	role	name
1	1	家老	吉田東洋
2	1	郷士	坂本龍馬
3	1	藩士	後藤象二郎
4	1	藩士	武市半平太
5	2	家老	小松帯刀
6	2	藩士	西郷隆盛
7	2	藩士	大久保利通
8	3	家老	木戸孝允
9	3	藩士	高杉晋作
10	3	藩士	伊藤博文

usersテーブル

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります

postsテーブル

このデータはどのテナントのものかわかるようになった

テナント単位で情報をとってくる

id	tenant_id	role	name
1	1	家老	吉田東洋
2	1	郷士	坂本龍馬
3	1	藩士	後藤象二郎
4	1	藩士	武市半平太
5	2	家老	小松帯刀
6	2	藩士	西郷隆盛
7	2	藩士	大久保利通
8	3	家老	木戸孝允
9	3	藩士	高杉晋作
10	3	藩士	伊藤博文

usersテーブル

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります

postsテーブル

他のテナントのデータを関連付けられない、
という制約はまだない

（あくまで、このテナントだよ〜としか指し示せてない）

テナント単位で情報をとってくる

usersテーブル

id	tenant_id	role	name
1	1	家老	吉田東洋
2	1	郷士	坂本龍馬
3	1	藩士	後藤象二郎
4	1	藩士	武市半平太
5	2	家老	小松帯刀
6	2	藩士	西郷隆盛
7	2	藩士	大久保利通
8	3	家老	木戸孝允
9	3	藩士	高杉晋作
10	3	藩士	伊藤博文

postsテーブル

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります

テナント単位で情報をとってくる

```
INSERT INTO posts (tenant_id, user_id, title, body)
VALUES ( 1, 6, '連絡事項', 'ほかのテナントに入っちゃった！');
```

tenant_id = 1 → 土佐藩
user_id = 6 → 西郷隆盛（薩摩藩）

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります
7	1	6	連絡事項	ほかのテナントに入っちゃった！

#52 [マルチテナント][確かめよう]
tenant_id だけでは防げない

テナント単位で情報をとってくる

```
FOREIGN KEY (user_id) REFERENCES users(id)
```

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります
7	1	6	連絡事項	ほかのテナントに入っちゃった！

DBが確認しているのは
「user_id = 6 が存在するか」だけ

テナント単位で情報をとってくる

```
FOREIGN KEY (tenant_id, user_id)
REFERENCES users(tenant_id, id)
```

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります
7	1	6	連絡事項	ほかのテナントに入っちゃった！

そのテナントに、そのユーザーが存在するか
まで確認ができるようになっ

#53 [マルチテナント][書いてみよう] 複合外部キーでテナントをまたぐ参照を防ぐ

テナント単位で情報をとってくる

```
FOREIGN KEY (tenant_id, user_id)
REFERENCES users(tenant_id, id)
```

id	tenant_id	user_id	title	body
1	1	2	藩政について	意見を共有します
2	1	3	会合のお知らせ	明日の会合について
3	2	6	連絡事項	藩士各位への連絡です
4	2	7	今後の方針	方針について相談します
5	3	9	集会について	次回の集会のお知らせです
6	3	10	情報共有	共有事項があります
7	1	6	連絡事項	ほかのテナントに入っちゃった！

そのテナントに、そのユーザーが存在するか
まで確認ができるようになっ

#53 [マルチテナント][書いてみよう] 複合外部キーでテナントをまたぐ参照を防ぐ

これでマルチテナントのDB設計の要所は抑えられる！



マルチテナントシステムで
本人か確認する

認証について

いろいろな認証・認可方式がでてきた
それを支えるDB設計について

パスワードを用いた認証

```
CREATE TABLE users (
  id BIGINT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(255),
  password_digest VARCHAR(255)
);
```

パスワードを持つ方式は
シンプルといえばシンプル

パスワードを用いた認証

[ブラウザ]
1. メールアドレス / パスワードを送る
　　↓
[アプリケーションサーバー]
2. users テーブルを見て本人確認
　　↓
[データベース]
users / id / email / password_digest

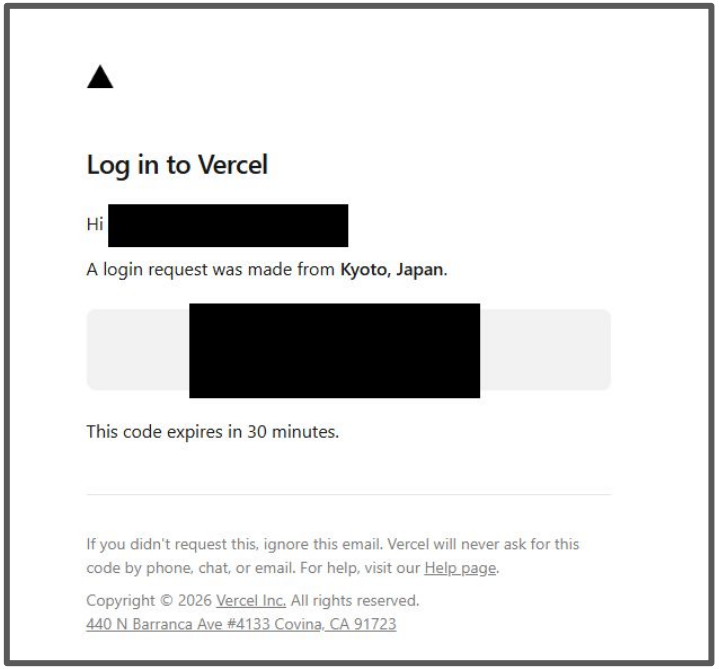
パスワードを用いた認証

[アプリケーションサーバー]
3. ログイン成功
4. Cookie を発行して返す
    ・この中にはuser_idを特定できる情報を埋め込んでる実装が多い
    ↓
[ブラウザ]
5. Cookie を保存
6. 次回以降のリクエストで自動送信
    ↓
[アプリケーションサーバー]
7. Cookie を見てログイン中のユーザを判定

認証について

パスワードを使わない認証方式がふえてきた

パスワードレス認証





ワンタイムパスワード

マジックリンク

これらを支える設計を考える

認証について（マジックリンク認証）



1.メールを入力

認証について（マジックリンク認証）



Mitsubachi <no-reply@mitsubachi.shiosalt.com>
To 自分

Mitsubachiへのログインリンクのご案内

Mitsubachiへのログインリクエストを受け付けました。

以下のリンクを開いてログインしてください。

Mitsubachiにログインする

ボタンが利用できない場合は、以下のURLをブラウザへ貼り付けてください。

http://localhost:5173/auth/verify?token=nLkmqwOJ1Th0GeNTFaX1-mxuu4YfKXY8AwiVKluAZuo&purpose=login

対象組織:

- Local Organization
- Login Mail Test Organization

発行日時: 2026年9月22日 21:06 JST

有効期限: 2026年9月22日 21:21 JST

開発環境なので大丈夫

2.ログインリンクがメールに届く / 開く

認証について（マジックリンク認証）



ログイン要求



トークン送付

トークン作成

トークンつき
リンクを開く

トークン照合

認証について（Email OTP認証）



ログイン要求



OTP送付

OTP作成

OTPを
入力・送信

OTP照合

認証について

ユーザーに送付 ~ 検証まで
サーバー側で保持しておかなければならない

インメモリDB / RDB どちらに保存する選択肢もあり

認証について

何を保存するべきか考える



ログイン要求



トークン/OTP送付

トークン作成

トークンそのもの

トークンつきリンクを開く
/OTP入力

トークン照合

有効期限

#54 [認証][書いてみよう] マジックリンク認証の状態を保存する
#55 [認証][書いてみよう] Email OTP の状態を保存する

マジックリンク認証

```
CREATE TABLE magic_link_authentications (
  id BIGINT PRIMARY KEY,
  user_id BIGINT NOT NULL,

  token_digest VARCHAR(255) NOT NULL,

  expires_at DATETIME(6) NOT NULL,
  used_at DATETIME(6) NULL,
  created_at DATETIME(6) NOT NULL,

  FOREIGN KEY (user_id)
    REFERENCES users(id)
);
```

OTP認証

```
CREATE TABLE otp_authentications (
  id BIGINT PRIMARY KEY,
  user_id BIGINT NOT NULL,

  code_digest VARCHAR(255) NOT NULL,

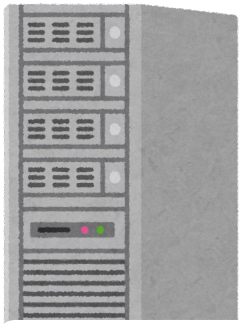
  expires_at DATETIME(6) NOT NULL,
  used_at DATETIME(6) NULL,
  attempt_count INT NOT NULL DEFAULT 0,

  created_at DATETIME(6) NOT NULL,
  FOREIGN KEY (user_id)
    REFERENCES users(id)
);
```

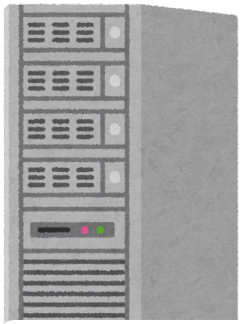


マルチテナントサービス
データベースの
高速化をしよう

どこまで共通化するか？

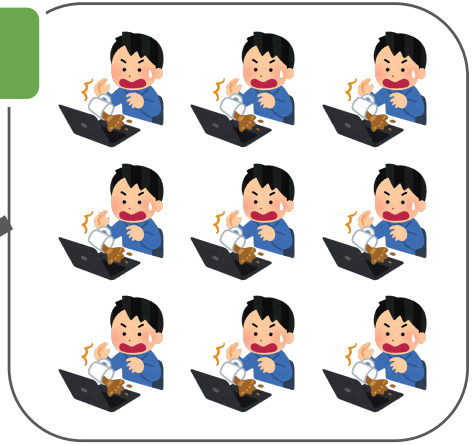


DBサーバー



アプリケーション
サーバー

テナント





テナント

複数のテナントがアクセス
高速に捌く必要がある！

そもそもインデックスとは

DBが行を速く探すための「検索用の索引」

```
CREATE INDEX idx_posts_tenant_id ON posts(tenant_id);
```

インデックス名

対象

#56 [インデックス][書いてみよう] テナント内の投稿検索を高速化する

そもそもインデックスとは

性能計測

行数	実行時間	VM_STEP	FULLSCAN_STEP	ソート	自動index行数	DBサイズ	データ生成	実行計画
1,000	0.09 ms	3,077	999	0	0	0.07 MB	5.7 ms	SCAN posts
10,000	1.21 ms	30,707	9,999	0	0	0.62 MB	21.5 ms	SCAN posts
50,000	4.75 ms	153,507	49,999	0	0	3.18 MB	88.5 ms	SCAN posts
100,000	8.07 ms	307,007	99,999	0	0	6.39 MB	170.1 ms	SCAN posts

性能計測 完了

行数	実行時間	VM_STEP	FULLSCAN_STEP	ソート	自動index行数	DBサイズ	データ生成	実行計画
1,000	0.04 ms	108	0	0	0	0.08 MB	6.9 ms	SEARCH posts USING INDEX idx_posts_tenant_id (tenant_id=?)
10,000	0.56 ms	1,008	0	0	0	0.72 MB	35.6 ms	SEARCH posts USING INDEX idx_posts_tenant_id (tenant_id=?)
50,000	2.46 ms	5,008	0	0	0	3.66 MB	176.1 ms	SEARCH posts USING INDEX idx_posts_tenant_id (tenant_id=?)
100,000	3.35 ms	10,008	0	0	0	7.40 MB	368.6 ms	SEARCH posts USING INDEX idx_posts_tenant_id (tenant_id=?)

複数のカラムでインデックスを張る

```
CREATE INDEX idx_users_tenant_role
                ON users(tenant_id, role);
```

マルチテナントサービスでは
「tenant_id + なにか」でインデックスを張ることが多い

#57 [複合インデックス][書いてみよう] テナント内の新着投稿を高速化する

複数のカラムでインデックスを張る

```
CREATE INDEX idx_users_tenant_role
                ON users(tenant_id, role);
```

マルチテナントサービスでは
「tenant_id + なにか」でインデックスを張ることが多い

とはいえ、なんでも貼ればいいというわけではないのが難しいところ・・・
（検索は早くなるが、挿入が遅くなる）

#58 [インデックス][考えてみよう] インデックスは多いほどよい？

複数のカラムでインデックスを張る

「どんな検索を早くしたいか」
から貼るインデックスを決める

まとめ

1. テナント境界は複合キーでDBにも守らせる
2. 認証では「秘密・期限・使用状態」を管理する
3. インデックスは実際の検索条件から設計する

没スライド集

単一の主キー/外部キー

id	name
1	土佐藩
2	薩摩藩
3	長州藩

id	tenant_id	role	name
1	1	郎士	坂本龍馬
2	1	藩士	後藤象二郎
3	1	藩士	武市半平太
4	2	藩士	西郷隆盛
5	2	藩士	大久保利通
6	2	家老	小松帯刀
7	3	藩士	木戸孝允
8	3	藩士	高杉晋作
9	3	藩士	吉田松陰

```
FOREIGN KEY (tenant_id) REFERENCES tenants(id)
```